



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC3413 — Implementación de Sistemas de Base de Datos — 1° 2026

LABORATORIO 2

Laboratorio: Índices.
Publicación: Lunes 13 de abril.
Ayudantías: Viernes 10 & Viernes 17 de abril.
Entrega: **Miércoles 22 de Abril hasta las 23:59 horas.**

Descripción del laboratorio

El objetivo de este laboratorio es completar la implementación de un *Linear Hash Index* en IIC3413DB [1]. Para esto, en [1] hemos implementado varias partes requeridas para el funcionamiento correcto del índice, y ustedes deben implementar unos métodos que quedaron incompletos. El código que deben estudiar para este laboratorio se encuentra en la carpeta `/src/storage/linear_hash_index/`.

Linear Hashs y su implementación en IIC3413DB

IIC3413DB cuenta con un Linear Hash Index, el que no guarda las tuplas mismas, sino los punteros a dónde se encuentran las tuplas en el `HeapFile` implementado en el primer laboratorio. Más preciso, para una tupla $t = (t_1, \dots, t_n)$ identificada en `HeapFile` con el `RID = (pageNum, dirSlot)` un Linear Hash Index sobre el atributo j almacena el triple $(\text{hash}(t_j), \text{pageNum}, \text{dirSlot})$ en uno de sus buckets.

A continuación explicaremos en detalle cómo funciona Linear Hash Index en IIC3413DB.

Implementación del Linear Hash Index. La clase `HashIndex` contiene la implementación de nuestro índice y es una implementación de la interfaz `Index` (ver `index.h`). Principales componentes de la clase son:

- `const HeapFile& heap_file`, el puntero al `HeapFile` dónde se encuentran nuestros datos;
- `const int key_column_idx`, la columna indexada por nuestro índice (nombre y el tipo en `heap_file`);
- `const FileId dir_file_id`, el archivo con páginas del directorio;
- `const FileId buckets_file_id`, el archivo con páginas de buckets de nuestro índice; y
- `N`, el número de buckets iniciales (al iniciar el índice sin datos);
- `depth`, la función de hash actual; quiere decir que se usa el valor $\text{hash}(\text{key}) \% (2^{\text{depth}} \cdot N)$ para calcular el bucket correspondiendo a key ;
- `split_pointer`, el número de siguiente bucket que hará el split;
- `dir`, el puntero a primera página del directorio.

Para crear un `HashIndex`, la clase requiere recibir un `HeapFile` en su constructor:

```
HashIndex(HeapFile& heap_file, int key_col_idx, FileId dir_file_id, FileId buckets_file_id, int32_t N)
```

dónde `key_col_idx` especifica la columna en la cual se encuentra nuestra llave de búsqueda, y `N` el número de buckets iniciales. Al llamar el constructor con un `HeapFile`, se generará un directorio (cual queda de esta misma forma por todo el ciclo de vida de este índice) cuyas páginas serán guardadas en el archivo `dir_file_id` y las páginas de las hojas, las cuales se guardarán en el archivo `buckets_file_id`.

Cada key en nuestro índice será un `Value` (valor de un atributo en un `Record` de la tabla). Por otro lado, los data entries en nuestro índice son de la forma `(hash(key), pageNum, dirSlot)`, dónde:

- `hash(key)` es un `int64_t` cual codifica el valor de llave de búsqueda (i.e. del valor de atributo indexado para nuestra tupla) usando una función de hash cual nos permite codificar cualquier valor en 8 bytes,
- `(pageNum, dirSlot)` es el RID de la tupla indexada en nuestro heap file. Aquí ambos `pageNum` y `dirSlot` son de 4 bytes. *Aquí no necesitan preocuparse si esto es la última versión de la tupla, dado que el índice es agnóstico a esto y simplemente indexa RIDs.*

Las entradas del tipo `(hash(key), pageNum, dirSlot)` las llamaremos *records del Hash Index* y los manejaremos como objetos de la clase `HashIndexRecord` definida en `HashIndex`.

Ahora explicaremos la estructura de las páginas del directorio y de los buckets.

Estructura de buckets. Un bucket es simplemente una lista ligada de páginas compuesta de la página principal (cabeza de la lista) y una serie de páginas de overflow. Los data entries de las páginas de buckets son triples `(hash(key), pageNum, dirSlot)`. La estructura de las páginas de buckets es la siguiente:



En la imagen `k = numTuples-1`. Especificación de los campos es la siguiente:

- `numTuples`: 8 bytes y codifica el número de data entries almacenados en esta página.
- `overflow`: El puntero a la siguiente página de este bucket. Son 8 bytes y en caso de contener valor -1 significa que no existe siguiente página en la lista de overflow. Si se trata de un número positivo este significa en número de la página en el archivo almacenando páginas de buckets (identificado por `buckets_file_id`). El número de la página actual y de su página de overflow no tienen ninguna conexión (e.g. no son números consecutivos).
- `di`: Los data entries de tipo `(hash(key), pageNum, dirSlot)`.

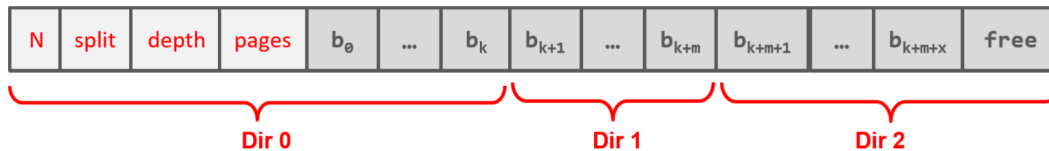
Es importante entender que los data entries vienen perfectamente empaquetados – quiere decir que no hay espacio libre entre dos data entries. En otras palabras cuando `numTuples = k` siempre hay precisamente `k` data entries que empiezan después de overflow pointer.

En la clase `BucketPage` pueden encontrar la implementación de las páginas de los buckets del nuestro índice. Para acceder o definir los metadatos de un `BucketPage` deben usar los siguientes métodos:

- `size_t get_tuple_count()` – retorna el número `numTuples`.
- `void set_tuple_count(int64_t new_tuple_count)` – actualiza el valor de `numTuples` a `new_tuple_count`.
- `size_t get_overflow_pointer()` – retorna el número `overflow`.
- `void set_overflow_pointer(int64_t new_overflow_pointer)` – actualiza el valor de `overflow` a `new_overflow_pointer`.

Sobre la capacidad de una página de bucket: La clase `BucketPage` tiene definido el valor `MAX_VALUES_PER_BUCKET` que define el máximo número de `numTuples`. La máxima capacidad física para páginas de IIC3413DB es 255.

Estructura del directorio. Las páginas del directorio en nuestro índice tienen una estructura muy simple y su objetivo principal es permitir acceso a la primera página de cada bucket. Efectivamente, nuestro directorio se puede pensar cómo un arreglo grande (paginado dentro de un archivo) que contiene unos pocos metadatos en la primera página y luego cada offset corresponde a puntero a la primera página del bucket correspondiente (identificada con su page number). La imagen abajo muestra un directorio con 3 páginas:



La interpretación de diferentes campos aquí es la siguiente:

- **N**: 4 bytes representando el número buckets iniciales de nuestro índice. Este número se asigna al crear el índice y no cambia nunca.
- **split**: 4 bytes representando la posición del split pointer en el estado actual del índice.
- **depth**: El número *depth* señala que la función actual de hash en uso es $hash(k) \% (2^{depth} \cdot N)$. 4 bytes.
- **pages**: La lista ligada de páginas libre para usar que se actualizan con el garbage collector explicado abajo. Por ahora piensen que es un número de 4 bytes.
- **b_i**: pageNum de la primera página del bucket *i* (a este número nos referimos con puntero a la página *i*). 4 bytes.

Observen aquí que solo la primera página del directorio (cual se queda pineada en el buffer durante el funcionamiento del índice) contiene los metadatos. Las páginas demás contienen solo punteros a la primera página del bucket correspondiente.

¿Cual es el número de buckets actuales? Este número lo pueden calcular con $N \cdot 2^{depth} + split$.

En la clase `HashIndexDir` pueden encontrar la implementación de las páginas del directorio. Para acceder o definir una de las posibles entradas la página del directorio la clase provee los métodos `get` o `set`, pero en este laboratorio ustedes no necesitarán usar estos métodos, dado que el directorio viene implementado por completo. El método que sí van a usar es:

```
int32_t get_bucket_page(size_t idx)
```

este método devuelve el pageNum de la primera página en el bucket número `idx`. Por ejemplo, si llamamos `get_bucket_page(7)` esto nos devolverá el número `b7` arriba, quiere decir, el número de la primera página del octavo bucket (cómo siempre, contamos desde 0).

Operaciones del Hash Index

Las operaciones fundamentales para el funcionamiento del índice son: la creación inicial del índice en base de un `HeapFile`, inserción de tuplas a un índice ya creado, eliminación de tuplas, y búsqueda. Cómo ya especificamos, para crear el índice, la clase `HashIndex` provee el método:

```
HashIndex(HeapFile& heap_file, int key_col_idx, FileId dir_file_id, FileId buckets_file_id, int32_t N)
```

cual crea el índice en base del `heap_file` existente. Acuerden que un índice siempre indexa a un `HeapFile`, lo qué quiere decir que todas las operaciones deben ser propagadas del `HeapFile` al índice, y no vice versa. Por lo

tanto, ustedes siempre pueden asumir que las operaciones que necesitan ejecutar (insertar o eliminar tuplas) será definido por el HeapFile y que el catalogo va a orquestrar todas las operaciones de manera adecuada. Por lo tanto, en lo que sigue, pueden asumir que la descripción de cada método de nuestro índice viene acoplado con las operaciones del HeapFile. En particular, cuando se inserta un record al índice, este ya fue insertado al HeapFile, y existe su RID, o si borramos un record identificado por su RID, este existe en el HeapFile y será eliminado también. Si revisan la implementación de este constructor pueden ver que el `heap_file` se solo asigna. Recuerden que el catálogo del sistema (`/src/system/catalog.cc`) es la componente encargada de crear el índice y insertar las tuplas del `heap_file` al índice. (*tl;dr implementen lo que dice abajo y no se preocupen si se usa de otra manera*).

¿Cómo se ve el Hash Index vacío? Esto depende de número de los buckets iniciales, pero en general se creará la primera página del directorio y una página de buckets para cada bucket inicial. Por ejemplo, si el número de buckets iniciales N es 100, se creará un único directorio y 100 páginas en el archivo de buckets, una para cada bucket inicial. Dependiendo del número N, pueden existir varias páginas del directorio (pero necesitan alrededor de 1020 buckets para pasarse a la siguiente página).

Insertión de tuplas. Para insertar los records usamos el siguiente método de la clase `HashIndex`:

```
void insert_record(RID rid)
```

Para insertar el record identificado con `rid` pueden asumir que este ya existe en el `HeapFile`. El flujo de inserción es siguiente:

- Primero, buscamos `rid` en el `HeapFile`, para conseguir su llave de búsqueda; i.e. el valor del atributo indexado (este valor lo denotamos con `k`).
- Segundo, calculamos `hash(k)` y con este valor obtenemos el número de la primera página del bucket que necesitamos acceder (ver `HashIndex::insert_record` – aquí usamos el directorio).
- Tercero, recorremos la cadena de páginas correspondiendo a nuestro bucket en orden y insertamos el data entry a la primera página con espacio para almacenarla.
- Si ninguna página correspondiendo a nuestro bucket cuenta con espacio suficiente, se crea una página de overflow nueva, el data entry se inserta a esta página, y la página se agrega a la cadena de páginas de overflow (usando el método `set_overflow_pointer`).

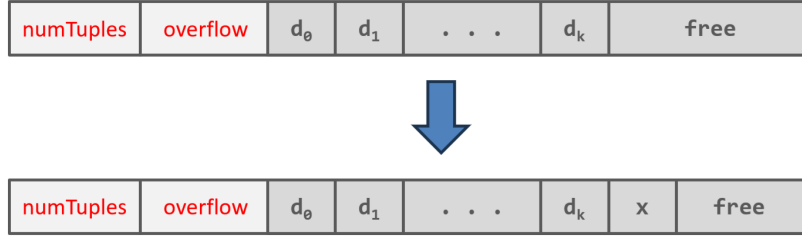
Ahora explicaremos los métodos de la clase `BucketPage` para insertar la tupla y el manejo de páginas de overflow mediante la clase `HashIndex`.

Insertando un data entry a la página de bucket: Para manejar la inserción de data entries a una página, la clase `BucketPage` provee el método:

```
bool Bucket::try_insert_record(uint64_t hash, const RID rid)
```

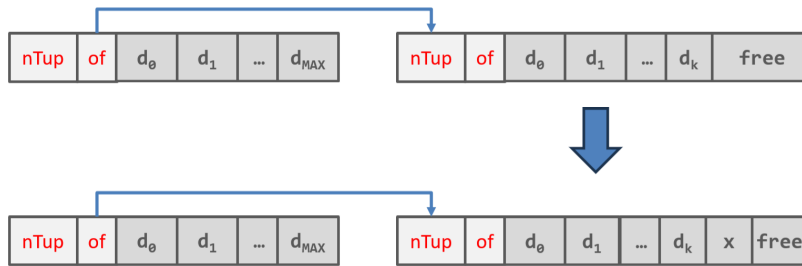
cual intenta insertar el data entry (`hash, rid=(pageNum,dirSlot)`) a su página.

Para esto, primero validamos que esta `BucketPage` cuenta con espacio para almacenar el data entry. Esto pueden validar con `get_tuple_count() < MAX_VALUES_PER_BUCKET`. En este caso deben insertar la entrada *al final de la página* y actualizar el número de tuplas. La clase `BucketPage` tiene métodos helper permitiendo hacer todo esto (en particular `set_record` y `set_tuple_count`). Este caso es ilustrado en la siguiente imagen (data entry insertado lo denotamos con `x`):



En la página modificada de nuestra imagen cambia la última entrada *y el `numTuples`* (cual se incrementa por uno). El método `try_insert_record` retorna `true` en este caso. Cuando no hay espacio suficiente el método retorna `false`.

¿Qué hacer si la tupla no cabe en la primera página del bucket? Aquí hay dos casos. Primer caso es cuando *hay una cadena de las páginas de overflow*, pero quizás en la primera página del bucket no hay espacio para la nueva entrada. En este caso se recorre la cadena usando los punteros de overflow (son igual a -1 al llegar a final de la cadena) y busca *la primera página con espacio*. En caso de encontrar una página con espacio se inserta a esta página igual que en el caso arriba. La ilustración de esto podemos ver en la siguiente imagen:



Cómo un hint, cuando están en el contexto de un `BucketPage current`, el número de la página overflow pueden conseguir con `auto next_page = current.get_overflow_pointer();`. Con esto pueden instanciar la siguiente página de overflow en la cadena escribiendo `BucketPage next(hash_index, next_page);`.

El segundo caso ocurre que al recorrer la cadena de nuestro bucket ninguna de las páginas cuenta con espacio para insertar la nueva tupla. En este caso es necesario crear una nueva página de overflow y encadenarla con la última página de la cadena actual. La siguiente imagen muestra esto:



La nueva página de overflow se puede pedir ocupando `next_overflow = get_new_bucket_page();` En este caso el índice ya tiene un mecanismo para entregarles una página nueva y ustedes no necesitan preocuparse de cómo se crea esta página. Noten que el valor arriba es el *numero de la página* y no la página misma. La página la pueden instanciar cómo explicamos en el caso uno arriba. Finalmente, recuerden que deben poner el overflow pointer de la última página de la cadena a `next_overflow`.

Split. En la implementación (incompleta) de `insert_record` pueden observar que esta tiene una variable booleana `new_overflow_page` cual debe ponerse a `true` solo en el caso que *creamos una nueva página de overflow* y va a señalar a nuestro índice que debe realizar un split del bucket apuntado por `split_pointer`. Quiere decir, que en nuestra implementación no hacemos un split al acceder a una página de overflow con

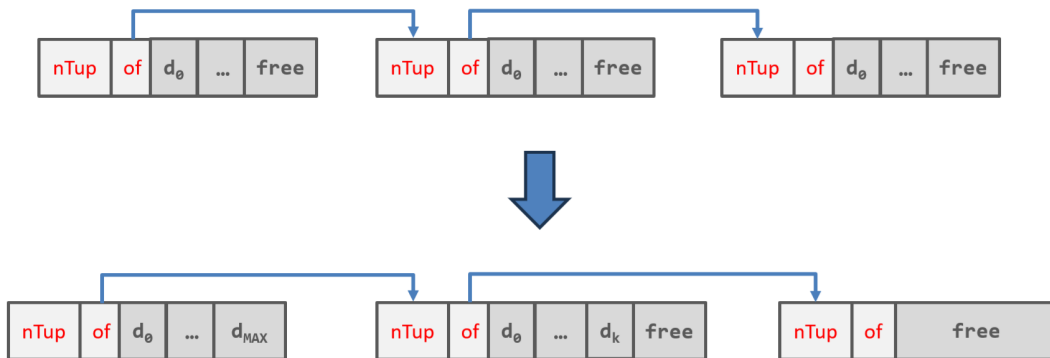
espacio, cómo explicamos en las clases, sino solo cuando *creamos una nueva página de overflow*. Este cambio se debe a una política más suave de splits, que permite manejar datos sesgados mejor que con splits agresivos.

En breve, la implementación de `HashIndex::insert_record` en `hash_index.cc` debe tener `new_overflow_page` puesto en `true` solo en el caso que `insert_record()` genera una nueva página de overflow, en cual caso hay que realizar un `split()`. Este método de la clase `HashIndex` funciona de siguiente manera:

- Primero, se revisa si es necesario cambiar el `depth` – el parámetro que nos dice cómo ocupar nuestro valor de hash. En caso de cambio el `split_pointer` vuelve a 0 (si revisan la implementación no se confundan con la lógica – estamos contando desde el 0).
- Segundo, conseguimos `split_bucket_page_number` – el bucket que hará el split. Similarmente, conseguimos la primera página del nuevo bucket llamado `next_bucket_page_number`.
- Tercero, se llama `redistribute(split_bucket_page_number, new_bucket_page_number)` cual redistribuye los valores del bucket entre el bucket actual y el bucket nuevo. Noten que esto puede crear páginas de overflow en el bucket nuevo. También, este método lo explicaremos en más detalles abajo, dado que debe dejar el bucket “limpio” para hacer garbage collection de páginas sin entradas.
- Cuarto, se recolectan la páginas vacías del bucket que hizo split, y agregan a una lista de páginas por utilizar al necesitar páginas nuevas. Esto se hace principalmente porque las páginas vacías no se sacan del archivo y pueden causar una crecimiento innecesario del mismo. Este mecanismo viene completamente implementado y lo pueden revisar en `HashIndex` y `HashIndexDir`.

Redistribución de valores usando `HashIndex::split()`. El método `split()` de la clase `HashIndex` itera por las páginas del bucket que hace el split y redistribuye sus valores según la función de hash actual, o mejor dicho según el `depth` actual y el `depth+1`. El método hace lo siguiente:

- Se recorre el bucket que hace el split (página por página) y se rehashan sus entradas con `depth+1` y con `depth` (quiere decir que se calcula $\text{hash}(\text{key}) \% (2^{\text{depth}+1} \cdot N)$ y con `depth` original).
- Si los dos valores no coinciden, hay que insertar la entrada al bucket nuevo creado por el split.
- Los entries que movimos se eliminan del bucket haciendo el split.
- Finalmente, las entradas que quedaron en el bucket que hace el split se mueven hacia las páginas iniciales. Quiere decir que si todas las entradas que quedan caben en la primera página, se moverán allá y las otras páginas se dejarán vacías (para esto basta poner su número de tuplas en 0). La siguiente imagen muestra cómo debe quedar el bucket que hizo split al final del proceso.



Observación: Es importante entender que dado que vamos a implementar eliminación de data entries, puede ocurrir que no es necesariamente que solo la última página de overflow tenga espacio libre. Por lo tanto nuestra redistribución debe considerar esto cómo ilustramos en la imagen arriba. Quiere decir, las tuplas que

se quedan en el bucket deben llenar las páginas iniciales del bucket a lo máximo. *Similarmente, las tuplas en el bucket nuevo, creado en el split, deben estar “empaquetadas igual!”*

Eliminación de tuplas. La eliminación de data entries en `HashIndex` simplemente busca si un data entry existe en el índice y en caso de encontrarlo en una página del bucket correspondiente, lo elimina, liberando el espacio y moviendo la última entrada en esta página a su posición. El método de la clase `HashIndex` es:

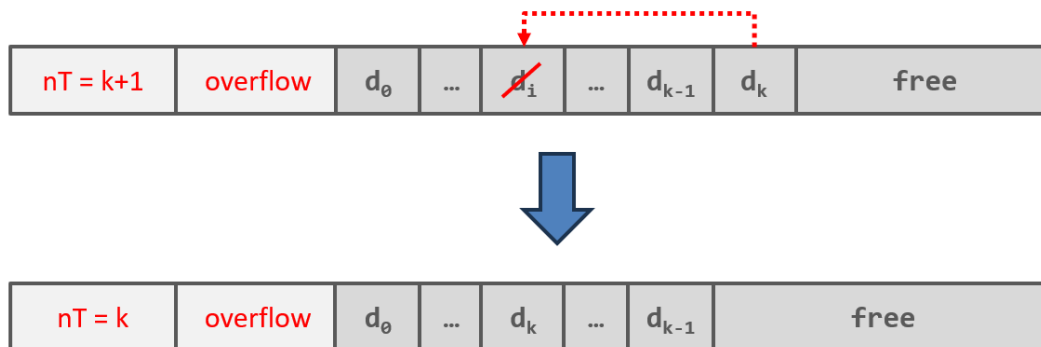
```
void delete_record(RID rid)
```

Este método primero consigue el valor del atributo indexado (la llave de búsqueda) similarmente cómo cuando llamamos `insert_record`. Luego *recorre la cadena de páginas del bucket correspondiendo a este valor* y aplica el método `delete_record(rid)` de la clase `BucketPage` a cada página en esta cadena.

El método `delete_record(rid)` de la clase `BucketPage` funciona de la siguiente manera:

1. Se recorren todas las entradas de la página del bucket.
2. Si alguna de estas entradas guarda el RID igual a `rid`, esta se elimina *y en su posición se coloca la última entrada en la página* (en caso de eliminar la última entrada no se pone nada).
3. Se actualiza el número de data entries en la página.

La siguiente imagen muestra cómo se ve una eliminación del data entry i en un `BucketPage`:



Es importante entender que la cadena de overflow se recorre por completo por dos razones: (i) porque la entrada quizás no está en la primera página; (ii) porque quizás tenemos dos data entries con el mismo RID. Lo segundo no debe ocurrir, pero en estricto rigor nuestro índice sí lo permite.

Cómo una aclaración, noten que mover la última entrada a la posición de la entrada eliminada es una forma de mantener las páginas de los buckets “empaquetadas”. Finalmente, recuerden que liberar el espacio de la última entrada no necesariamente significa cambiar los bytes en la página, sino simplemente bajar el número de tuplas en la página, lo que hace que el espacio de esta última entrada quede libre para ser reutilizado por nuevas entradas (ver la implementación de `try_insert_record`).

Búsqueda. Finalmente, para poder buscar una tuple por el valor de columna indexada la clase `HashIndex` provee el método:

```
std::unique_ptr<RelationIter> get_iter(const Value& key)
```

Este método retorna un iterador permitiendo iterar sobre las tuplas cuales tienen el valor del atributo indexado igual a `key` – i.e. la razón de existencia de nuestro índice. Para este laboratorio no explicaremos mucho los detalles de su implementación, pero para ustedes puede ser útil para hacer testing de su solución. Para ver cómo se puede ocupar este iterador consulten tests en la carpeta `/src/bin/`.

Tareas para hacer en este laboratorio

Problema 1: Inserción a un bucket [1.5 puntos]. En la clase `HashIndex` el método:

```
void HashIndex::insert_record(RID rid)
```

tiene una implementación incompleta. Este método es encargado insertar el data entry correspondiendo a `rid` a su bucket. Como explicamos arriba hay dos casos importantes: (i) hay una página en la cadena de este bucket con espacio, en cual caso se inserta en la primera; y (ii) hay que crear una nueva página de overflow y encadenarla con este bucket. En particular, se les pide completar la implementación en el archivo `hash_index.cc` marcado como `TODO 1` cual trata el caso de inserción a un bucket. Noten que la implementación de inserción a una página de bucket ya viene implementada. Su trabajo es realizar correctamente la iteración sobre las páginas del bucket e insertar a correspondiente, o crear una nueva de ser necesario.

Problema 2: Eliminación de un data entry [1.5 puntos]. En este problema completarán la implementación del método `delete_record` de la clase `BucketPage`. En particular, necesitan agregar el bloque del código marcado como `TODO 2`. Para esto deben implementar ustedes según la lógica explicada arriba. Recuerden que (en teoría), un mismo `rid` puede ocurrir varias veces en una misma página, como en varias páginas de un mismo bucket. El método `delete_record` de la clase `HashIndex` depende crucialmente de este método para poder eliminar las entradas del índice.

Problema 3: Redistribución de elementos del bucket que hace el split [3 puntos]. En la clase `HashIndex`, para realizar la redistribución de elementos durante un split, el método `split` llama al método:

```
void HashIndex::redistribute(int split_bucket_page_number, int new_bucket_page_number)
```

Este método quedó sin implementación y ustedes la deben completar aplicando la lógica explicada arriba. El bloque de código por completar está marcado como `TODO 3`. Recuerden aquí que su redistribución debe hacer dos cosas: (i) mover entradas que corresponden desde el bucket que hace split al bucket nuevo; y (ii) “limpiar” el bucket que hace split, corriendo todas las entradas en las páginas del bucket hacia izquierda, manteniendo las páginas completamente “empaquetadas”. Las páginas que no tienen tuplas deben quedar con tuple count puesto en 0 para que el recolector de basura implementado en el método `HashIndex::split()` las puede reutilizar.

Bonus: bugs [0.2 puntos – se suma a otras tareas] Si encuentran un bug importante en la implementación de IIC3413DB lo pueden reportar y reciben este bonus. Nuestro código definitivamente no es perfecto y cada proyecto de software no-trivial probablemente contiene algunos bugs. Si encuentran algo que permitirá mejorar IIC3413DB su esfuerzo se premiará con este bonus cual puede, en teoría, subir su nota encima de 7.

Código

El código base está disponible en [1] en la rama `Lab2`. Asegúrense que están usando la última versión del sistema! Los tests se publicarán en el mismo repositorio en la carpeta `src/bin`. Como parte del código, también publicaremos la solución de Laboratorio 1.

Evaluación y entrega

El día límite para la entrega de esta tarea será el Miércoles 22 de Abril a las 23:59 horas. Para ello se utilizará el formulario de canvas. *Su entrega debe contener todo el código de la carpeta `src` y nada más, y debe compilar cuando se les agregan otras carpetas en el repositorio. El código de la carpeta `src` debe ser entregado como un archivo `.zip`.* Por último, la evaluación será en base a varios tests.

Ayudantía y preguntas

El día viernes 10 & Viernes 17 de abril se realizarán unas ayudantías donde se darán más detalles sobre IIC3413DB y el laboratorio. Para preguntas se pide usar issues del GitHub.

Referencias

[1] IIC3413DB. <https://github.com/IIC3413/2026-1>.